

LLM training infrastructure at scale

front: ten minutes a phase · off-bank, Cluster Infra insurance

Anthropic question this is: none of the six bank cards. It is the Cluster Infrastructure job itself (R1 lifecycle, R5 fault tolerance, R7 operations), and the likeliest shape if the interviewer pulls from their own backlog. **Reported cousins:** gang scheduler on a fixed GPU pool (AceOffer 3, prachub Aug 12) · Orava Senior "fault-tolerant training of a 100B+ model across thousands of nodes" with the follow-up *checkpoints now take 20 minutes* · Jobright "distributed training pipeline with fault tolerance" · prep-artifact Q9 and Q17. **Shares mechanism with:** P1.1 eight-GPU gang reservation · P3 base-model distribution (the hot pool is the NVMe cache). **Alex graded the clock:** net design time is 45–50 min, not 60.

1 REQUIREMENTS 0–8 min · five questions, then move

- **Ask:** pretrain, fine-tune or RL on one fleet? total GPUs and largest single job? one region, or may a job span sites? who submits, and the priority order? done = a registered model version?
- **A user can:** submit a job (base model, GPU count, dataset, priority) · start only when every GPU is ready · watch step, loss, throughput, failures · **get the trained model back, versioned** · cancel and resume · run many in parallel.
- **Technical challenges:** a job is a **gang**, partial start is worthless · one dead GPU kills the job, so effective MTBF falls linearly with job size · checkpoint needs a **barrier** · fabric decides placement, cross-island all-reduce taxed every step · idle GPU = money burned.
- **Anchors:** 20,000 GPUs · 8/node = 2,500 nodes · job 64–4,096 GPUs · 100B params × 16 B = ~1.6 TB state · single region + twin site.

2 CORE ENTITIES 8–16 min · generation on everything

- **Team**(quota, priority class) · **Job**(base model, gpus, dataset, epochs, priority, state, **generation**)
- **Allocation**(nodes, rank map, fabric island, generation) — created atomically or not at all
- **Node**(8 GPUs, fabric address, pool hot|cold|spare, resident base models, health)
- **Checkpoint**(epoch, step, per-rank shard manifest, URIs, **committed**) · **Dataset**(shards, seed, cursor)
- **Quota** · **HealthEvent**(node, gpu, class, ts)
- **Not entities, say so:** the GPU alone (a field of the node) · the epoch (a counter) · the weights (content of a checkpoint).

3 API 16–22 min · define semantics as you write

- `POST /jobs {base_model, gpus, dataset, epochs, priority, checkpoint_policy} → job_id`, idempotent on a client key
- `GET /jobs/{id} → state, epoch, step, allocation, last committed checkpoint, restarts`
- `POST /jobs/{id}/cancel · /resume {from_checkpoint} · GET /quotas/{team}`
- `GET /models/{name}/versions — the deliverable` (Alex had to prompt for this)
- Internal: `/nodes/{id}/health` · `/allocations/{id}/report {rank, generation, state, step}` · `POST /checkpoints` commits the manifest

4 DATA FLOW 22–26 min · one job end to end

- 1 post → 2 gateway auth → 3 quota admits → 4 persist + enqueue (queue absorbs bursts, ages so big jobs never starve) →
- 5 gang scheduler finds N healthy nodes **in one fabric island** that already hold the base model, reserves **atomically** with generation G; cannot find them all → the job **waits**, never half-starts →
- 6 agents verify GPUs, assign ranks, open the collective, barrier → 7 loop: read shard, fwd/bwd, all-reduce, optimizer step →
- 8 epoch boundary: barrier, per-rank snapshot VRAM → NVMe, **release the barrier**, async chunked upload, **manifest commits last** →
- 9 fault: agent emits → drain node → spare swaps in → bump generation (fences stale ranks) → restart from last committed checkpoint → 10 promote to the registry.

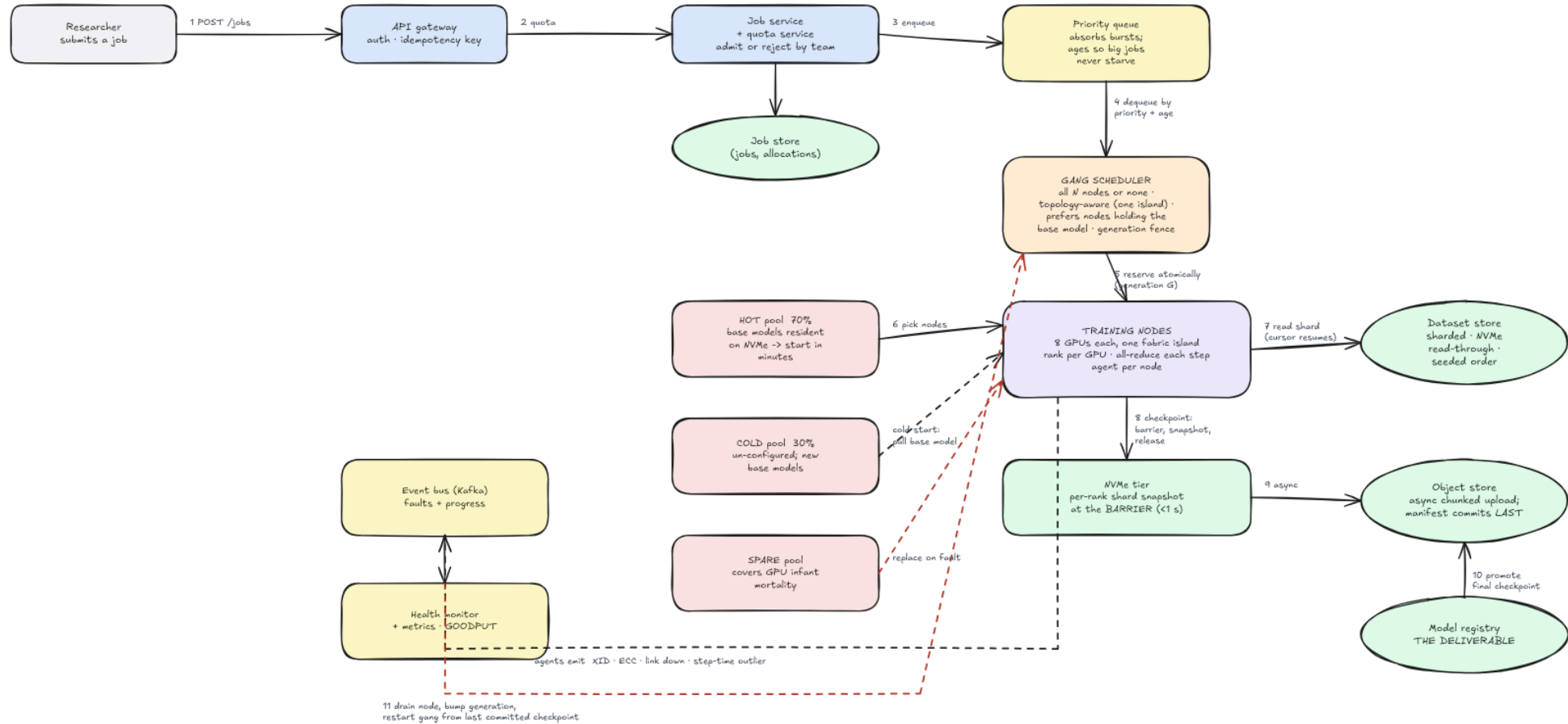
5 HIGH-LEVEL DESIGN 26–32 min · finish here, not at 40

- **Control:** researcher → gateway → job + quota service → job store → priority queue
- **Scheduling:** gang scheduler, topology-aware, hot/cold/spare pools, generation fence
- **Data plane:** training nodes, 8 GPUs, one fabric island, one agent each
- **Storage:** dataset (sharded, NVMe read-through, seeded order) · checkpoint NVMe → object store · **model registry = output**
- **Observability:** agents → event bus → health monitor; failure signal back to the scheduler
- **Sums:** 4,000-GPU job = 500 nodes; 6-month node MTBF → a failure every ~9 h; a 3-week run is interrupted ~50 times, so **restart cost is the design driver**. 400 MB/rank; VRAM→NVMe <1 s: the barrier is the cost, not the bytes. 60 s every 20 min = 5%; epoch-aligned < 2%.

6 DEEP DIVES 32–45 min · offer, then follow

- **Gang scheduling without starvation:** all-or-nothing, backfill small jobs into holes, age the queue, preempt only at a checkpoint boundary.
- **Topology:** NVLink in-node, rail-optimised in-island; splitting a job across islands taxes every all-reduce. Locality is a **hard constraint**.
- **Checkpointing (Alex's question):** barrier → per-rank shard to NVMe → release → async chunked upload → commit manifest last; keep N–1; checksum. **Epoch-aligned**, because researchers compare quality per epoch. "Takes 20 minutes?" only the barrier blocks — move bytes off the critical path, shard wider.
- **Failures:** fail-stop (off the bus, ECC, link down) → drain, swap, restart · fail-slow straggler → step-time outlier, one slow rank slows all 4,000 · silent corruption → gradient-norm outlier + shadow recompute.
- **Hot/cold pools:** 70% hot with frequent base models on NVMe (start in minutes), 30% cold. A cache; eviction by base-model frequency. Trade-off: capacity held for speed.
- **Goodput** = GPU-hours that committed steps ÷ GPU-hours allocated. One number catching stragglers, restarts and checkpoint overhead.
- **The bow at 45:** admit, queue, place all-at-once on topology-close nodes that already hold the base model, train, checkpoint at epochs, swap spares when hardware dies every few hours, output a registered version. Nothing on the training path waits on the control plane; no job ever half-starts.

The board at minute thirty-two LLM training infrastructure — the answer



Legend

control plane scheduling data plane storage queue / observability node pools

NUMBERS TO SAY OUT LOUD

20,000 GPUs / 8 per node = 2,500 nodes. A job is 64 to 4,096 GPUs.

4,000-GPU job = 500 nodes; node MTBF 6 months -> a failure every ~9 hours.

100B params x 16 bytes (weights + optimizer moments) = ~1.6 TB of state;

sharded 4,000 ways = ~400 MB per rank; VRAM -> NVMe at 5 GB/s is <1 s.

The BARRIER is the cost, not the bytes. The upload is off the critical path.

Checkpoint overhead: 60 s every 20 min = 5% of the fleet. Epoch-aligned (~1 h) < 2%.

Expected lost work on a crash = half the checkpoint interval.

Metric that exposes everything: GOODPUT = GPU-hours that committed steps / GPU-hours allocated.

RED: what fails and what you say

Fail-stop (GPU off the bus, ECC, link down): agent detects, drain, swap a spare, restart from checkpoint.

Fail-slow (straggler): detect by step-time outlier; one slow rank slows all 4,000.

Silent corruption: per-rank gradient-norm outlier + shadow recompute; answer was_it_hardware(job, steps).

Checkpoint on EPOCH boundaries, not a timer: researchers compare quality per epoch and restart from one.